



Reichman University

Efi Arazi School of Computer Science

M.Sc. Program in Machine Learning and Data Science

Final Project Report

Permuted Dogs versus Cats

Student 1 ID: 123456789 **Student 2** ID: 987654321

Advisors: Shuli Finley (Reichman University) — Alon Oring (Reichman University)

May 25, 2026

Abstract

This project studies binary Dogs versus Cats classification after fixed tile-wise permutations are applied to each image. The proposal frames the task as a relaxed image-domain analogue of permuted-puzzle problems with cryptographic motivation: instead of pixel-wise permutations over function graphs, images are divided into coarse grids and their tiles are rearranged before classification. We implement a notebook-first experimental pipeline around the Kaggle Dogs vs Cats dataset, train baselines on unpermuted and permuted images, evaluate augmentation and curriculum strategies, and define model-agnostic permutation hardness metrics. The updated experiment matrix uses four grid settings, 1 tile / none, 4-by-4, 7-by-7, and 10-by-10, crossed with three deterministic permutation functions named easy, medium, and large. Each remote run trains for 10 epochs, giving 4×3 grid-permutation records per model or ablation. Existing saved numeric tables are retained as prior outputs until the updated remote Colab sweep is rerun.

1 Introduction

Image classification models are typically evaluated on data whose spatial structure is in-

tact. This project asks what happens when that structure is deliberately disrupted before learning. Each input image is split into an $N \times N$ grid, the tiles are permuted by a fixed mapping, and the classifier must still distinguish dogs from cats. The task is motivated by a proposal for an intermediate experiment connected to permuted puzzles and cryptographic hardness [1]. The image setting relaxes the original cryptographic problem in two ways: it uses natural images rather than graphs of functions, and it applies tile-wise permutations rather than pixel-wise permutations.

The project has three main objectives. First, it measures how tile resolution affects the feasibility of classification after permutation. Second, it asks whether a neural network succeeds by learning or compensating for the permutation, or whether enough class-discriminative signal remains directly visible in the permuted images. Third, it develops ways to rank permutations according to their induced difficulty for a fixed dataset and tile count.

The implementation follows a reproducible, notebook-first workflow. Part 1 trains model baselines on unpermuted images and deterministic tile permutations at 4-by-4, 7-by-7, and 10-by-10 resolution. Part 2 tests improvement strategies for the same permuted setting, including image augmentations and curricula. Part 3 computes permutation hardness metrics and joins them to validation accuracy so that image-independent permutation properties can be compared against model performance.

2 Related Works

The motivating proposal cites permuted puzzles as a source of cryptographic hardness, where fixed permutations can make struc-

tured objects difficult to recover or classify [1]. This project does not attempt to solve the cryptographic problem directly. Instead, it uses the Dogs vs Cats setting as an empirical stepping stone for studying how much semantic information survives structured spatial disruption.

Convolutional neural networks are a natural starting point because they exploit locality and translational structure in images. ResNet architectures are especially relevant because residual connections improve optimization of deep convolutional models [3]. Since tile permutations partially destroy local adjacency while preserving tile content, ResNet-18 provides a useful baseline for testing whether standard convolutional inductive biases remain helpful.

The ablations also connect to standard regularization and robustness ideas in deep learning [2]. Random erasing, patch shuffling, same-label CutMix, and combined augmentations are used to expose the model to local occlusion or spatial disruption. Curriculum learning variants gradually control corruption probability or permutation difficulty, testing whether staged exposure to harder permuted inputs improves validation accuracy.

3 Data Overview

The dataset is Kaggle’s Dogs vs Cats image dataset, with labels inferred from filenames such as `cat.123.jpg` and `dog.456.jpg`. The repository expects labeled training images under `data/dogs-vs-cats/train`; the unlabeled Kaggle test folder is not used for validation accuracy. The task is binary classification, and the saved experiment outputs report validation accuracy after the configured training runs.

For the unpermuted setting, each image is

processed as a normal dog or cat image and used as the benchmark for post-permutation performance. For the permuted setting, each image is divided into a square tile grid and the same deterministic tile permutation is applied across images for a run. The updated grid settings are one tile / none, 16 tiles, 49 tiles, and 100 tiles. The one-tile baseline is repeated under the easy, medium, and large labels so that all parts use the same 4×3 grid-permutation matrix.

4 Methodology

The experimental pipeline is organized into three notebooks. Part 1 establishes baselines by training on the unpermuted task and on fixed deterministic tile permutations. Part 2 reuses the Part 1 ResNet-18 baseline and trains improvement ablations on the same task family. Part 3 computes permutation descriptors and correlates them with the validation accuracy observed for the corresponding deterministic records.

The primary model in the current results is ResNet-18 [3]. Experiments save raw run outputs, aggregated CSV summaries, and figures under **outputs/results** and **outputs/figures**. Aggregation reports mean final validation accuracy, mean best validation accuracy, standard deviation where multiple runs exist, and the number of runs.

4.1 Training and Validation Sets by Experiment

All experiments use the labeled Kaggle Dogs vs Cats training folder as the only source of supervised examples. Labels are inferred from filenames, and the samples are split once with a stratified class-balanced split using seed 42, validation fraction 0.2, and test frac-

Part	1	Training set	Validation set
run			
1 tile / none		20,000 unpermuted images repeated under easy, medium, and large labels.	5,000 unpermuted held-out images.
4-by-4 permutations		Same 20,000 images with easy, medium, and large deterministic 4-by-4 permutations.	Same 5,000 held-out images with the matching deterministic permutation.
7-by-7 permutations		Same 20,000 images with easy, medium, and large deterministic 7-by-7 permutations.	Same 5,000 held-out images with the matching deterministic permutation.
10-by-10 permutations		Same 20,000 images with easy, medium, and large deterministic 10-by-10 permutations.	Same 5,000 held-out images with the matching deterministic permutation.

Table 1: Part 1 training and validation sets. The same split is reused for ResNet-18, DeiT-Tiny, and MLP-Mixer Base, and each remote run trains for 10 epochs.

tion 0.0. For the full dataset this gives 20,000 training images and 5,000 validation images: 10,000 cats and 10,000 dogs for training, and 2,500 cats and 2,500 dogs for validation. The validation split is held out from optimization and is not modified by training-only augmentations such as random erasing, CutMix, patch shuffle, or curriculum-stage sampling.

The curriculum ablations in Part 2 use stage-specific training loaders, but they do not create separate held-out validation splits per stage. Across both curricula, validation re-

mains the same 5,000-image held-out loader for the final target run.

4.2 Deterministic Permutation Functions

The updated experiments no longer sample random tile permutations for the main matrix. Instead, every part uses the same three deterministic functions: easy, medium, and large. This makes comparisons across Parts 1, 2, and 3 reproducible and interpretable, because a given difficulty label refers to the same construction wherever it appears.

The easy function performs a small number of local adjacent swaps, preserving most local image structure. The medium function combines row and column shifts with several local swaps, creating moderate spatial disruption while retaining some regularity. The large function applies broad global reversals or rotations with additional swaps, producing the highest layout disruption. For the one-tile baseline, all three labels map to the unpermuted image; the labels are still saved so the experiment matrix remains exactly 4 grid settings by 3 functions.

4.3 Architecture Comparison and Frozen Backbones

To compare architectural inductive biases beyond ResNet-18, the Part 1 pipeline also supports three pretrained feature extractors: ResNet-18, DeiT-Tiny [5], and MLP-Mixer [4]. All three are used in the same transfer-learning regime: the ImageNet-pretrained backbone is frozen and only the final binary classification head is optimized. This makes the relevant training-capacity comparison the number of learnable head parameters, not the total number of parameters stored in the network.

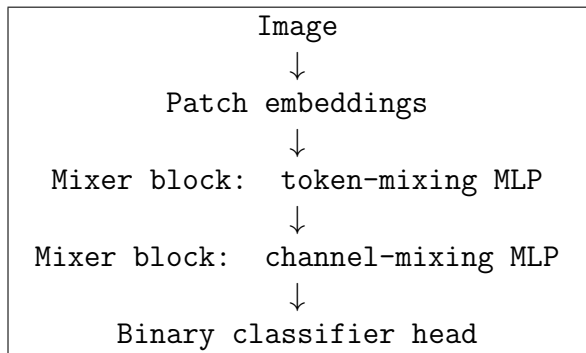


Figure 1: Simplified MLP-Mixer flow. Token mixing exchanges information across patches, while channel mixing recombines features within each patch representation.

The three models differ mainly in how they exchange information across the image. ResNet-18 uses convolutions, giving it a strong local-neighborhood bias. DeiT-Tiny represents the image as patches and uses self-attention to let patches interact. MLP-Mixer also operates on image patches, but replaces attention and convolutions with two MLP operations: token mixing, which combines information across patches, and channel mixing, which recombines feature dimensions inside each patch representation.

The MLP-Mixer choice required special care. The smaller `mixer_s16.224` architecture would have been closer in total parameter count, but the available `timm` configuration does not provide pretrained weights for that variant. Freezing an untrained MLP-Mixer backbone would train only a classifier on random fixed features, which would not be a meaningful comparison to pretrained ResNet-18 and DeiT-Tiny. We therefore use the larger pretrained `mixer_b16.224.goog_in21k_ft_in1k` model. Although this increases the number of frozen parameters and may increase forward-pass cost, the number of optimized parameters remains comparable to the other models: 1,538 learnable parameters for MLP-Mixer Base

versus 1,026 for ResNet-18 and 386 for DeiT-Tiny.

4.4 Improvement Strategies

Part 2 evaluates training strategies against the regular Part 1 ResNet-18 baseline. The main distinction is whether a strategy adds new training views of the same images, changes the optimization objective, or changes the order in which the model sees easy and hard inputs. In all cases, validation is performed on the held-out validation images with the target permutation and without training-only perturbations.

For the counts below, let B be the configured batch size and let $N_{\text{train}} = 20,000$ be the number of training images. The ablations do not append files to the dataset. Instead, they replace the ordinary batch tensor with an augmented view during the forward pass, so the stored increase is zero even when every processed training example is augmented.

4.4.1 Regular Part 1 Baseline

Main idea. This is the reference point for Part 2. ResNet-18 is trained normally on the target permutation, with no extra augmentation, no curriculum, and the standard loss.

Reason. The baseline is necessary because every ablation must be judged against ordinary training on the same task. Without it, an improvement strategy might look useful simply because the target permutation is easier than another run.

Deeper dive. The baseline uses the same train and validation split as Part 1. For a permuted run, both training and validation images are transformed by the same fixed tile permutation for that run. It has one training stage and does not increase the dataset size. If x_i is an input image, P_π is the target tile

permutation, and f_θ is the classifier, the baseline minimizes

$$\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \ell(f_\theta(P_\pi(x_i)), y_i).$$

The number of added augmented images is 0 per batch and 0 per epoch; the number of processed training views remains B per batch and 20,000 per epoch.

4.4.2 Patch Shuffle

Main idea. Patch shuffle randomly rearranges smaller patches during training. The model therefore sees extra local spatial disruptions in addition to the fixed tile permutation.

Reason. This is relevant because the main task already tests whether class information survives spatial reordering. If extra random patch movement helps, it suggests that robustness to local layout disruption is useful for permuted Dogs versus Cats classification.

Deeper dive. The target permutation is still the run-level task definition, but each training batch can also receive random patch shuffling. Let S_ρ denote a random within-image patch permutation sampled independently during training. The effective training view is

$$\tilde{x}_i = S_{\rho_i}(P_\pi(x_i)),$$

and the loss is computed on $\ell(f_\theta(\tilde{x}_i), y_i)$. In the implementation the shuffle probability is 1.0, so a batch of size B contains B shuffled views and an epoch contains 20,000 shuffled views. These views are not appended to the dataset; they replace the current batch examples. The validation loader does not use patch shuffle, so reported accuracy measures the fixed target validation permutation.

4.4.3 Random Erasing

Main idea. Random erasing hides random image regions during training. The model must learn from the remaining visible evidence rather than depending on one part of the animal.

Reason. Tile permutations can break global shape, so a model may need to rely on partial cues such as fur texture, ears, eyes, color, or background context. Random erasing tests whether forcing the model to survive missing local evidence improves that behavior.

Deeper dive. This method creates occluded variants of the same training examples on the fly. Let $M_i \in \{0, 1\}^{H \times W}$ be a sampled erasing mask and let v be the fill value used by the transform. The augmented image can be written as

$$\tilde{x}_i = M_i \odot P_\pi(x_i) + (1 - M_i) \odot v.$$

Random erasing is configured with probability 1.0 and scale range 0.02 to 0.20, so each batch contributes B erased training views and each epoch contributes 20,000 erased views. The stored dataset still contains only the original 20,000 training images. The validation set remains un-erased and is transformed only by the target permutation.

4.4.4 Same-Label CutMix

Main idea. Same-label CutMix replaces part of one training image with a patch from another image of the same class. The label stays unchanged because cat is mixed only with cat and dog only with dog.

Reason. This experiment asks whether more same-class local variation helps the model classify permuted images. It is especially relevant when global object layout is damaged and the classifier may need to combine local evidence from separated tiles.

Deeper dive. Unlike ordinary CutMix, this version avoids mixed labels. For image i , the batch augmentation samples a partner j with $y_j = y_i$ and a rectangular binary mask M_i . The hybrid view is

$$\begin{aligned} \tilde{x}_i &= M_i \odot P_\pi(x_i) \\ &\quad + (1 - M_i) \odot P_\pi(x_j), \\ \tilde{y}_i &= y_i. \end{aligned}$$

The batch-level CutMix probability is 1.0, so each batch attempts to create B same-label hybrid views. If an example has no same-label partner inside a particular batch, it is left unchanged; with shuffled binary batches this edge case is rare for normal batch sizes. The stored increase is 0, while the effective exposure is up to B hybrid views per batch and up to 20,000 per epoch. Validation uses the original held-out images and does not apply CutMix.

4.4.5 Combined Augmentations

Main idea. Combined augmentations apply more than one training perturbation in the same ablation. The goal is to expose the model to richer variation than any single augmentation provides.

Reason. Different augmentations stress different weaknesses: patch shuffle stresses spatial order, CutMix stresses partial object evidence, and image perturbations stress reliance on specific pixels. Combining them tests whether these robustness pressures are complementary.

Deeper dive. The training split remains the same 20,000 images, but the effective variety of examples increases because stochastic augmentations produce multiple possible views. In the implementation, combined augmentations apply random erasing in the image transform and then apply same-label CutMix followed by patch shuffle at the batch level.

One training view can be summarized as

$$\begin{aligned}\tilde{x}_i &= S_{\rho_i}(M_i \odot E(P_{\pi}(x_i)) \\ &+ (1 - M_i) \odot E(P_{\pi}(x_j))), \\ y_j &= y_i.\end{aligned}$$

where E denotes the image-level erased transform and S_{ρ_i} denotes the random patch shuffle. This produces B composite augmented views per batch and 20,000 per epoch, with 0 stored dataset growth. The validation set is not augmented; it remains the fixed held-out target-permutation set. This makes the comparison a test of whether richer training exposure improves ordinary target validation accuracy.

4.4.6 Focal Loss

Main idea. Focal loss changes the training objective so harder examples receive more emphasis and easy examples receive less.

Reason. Some permuted images may remain easy because local class cues are obvious, while others may become much harder after tile movement. Focal loss tests whether concentrating learning on those harder cases improves performance.

Deeper dive. This strategy does not add new images to the training set and has one stage. It changes only the loss used during optimization. For the binary target probability p_t assigned to the correct class, focal loss uses

$$\text{FL}(p_t) = -\alpha(1 - p_t)^{\gamma} \log(p_t),$$

with $\gamma = 2.0$ and $\alpha = 1.0$ in the default ablation configuration. The number of added augmented images is 0 per batch and 0 per epoch. Validation uses the same held-out target set, so any difference comes from the training objective rather than from extra data.

4.4.7 Corruption-Probability Curriculum

Main idea. The model starts with mostly intact images and gradually sees the target permutation more often. The corruption probability increases across four stages.

Reason. Jumping immediately into a hard permutation may make optimization unstable. This curriculum tests whether gradual exposure lets the model first learn dog-versus-cat cues and then adapt those cues to more frequent spatial corruption.

Deeper dive. Each stage uses the same 20,000 training images and target tile permutation, but changes the probability that the permutation is applied. For stage probability q_s , the sampled training view is

$$\tilde{x}_i = \begin{cases} P_{\pi}(x_i), & z_i < q_s, \\ x_i, & z_i \geq q_s, \end{cases}$$

$z_i \sim \text{Uniform}(0, 1).$

The stages use $q_s \in \{0.1, 0.3, 0.5, 0.8\}$, so the expected number of permuted views per stage epoch is $q_s N_{\text{train}}$: 2,000, 6,000, 10,000, and 16,000 respectively. In a batch of size B , the expected number of permuted views is $q_s B$. This does not store a larger dataset; it changes the sampled training view. There is no separate validation set for each stage; validation always uses the fixed target validation loader.

4.4.8 Permutation-Difficulty Curriculum

Main idea. The model starts on easier spatial structure and moves toward the final harder permutation. Instead of changing probability, it changes the tile grid difficulty across stages.

Reason. Smaller or easier permutations preserve more of the original image structure.

This experiment tests whether a model can learn useful class cues under mild disruption before facing the final, harder tile rearrangement.

Deeper dive. Each stage uses the same 20,000 training images but changes the tile grid used for training. If g_s is the stage grid size and π_s is the corresponding sampled or target permutation, then

$$\tilde{x}_i^{(s)} = \begin{cases} x_i, & g_s = 1, \\ P_{\pi_s}^{(g_s)}(x_i), & g_s > 1. \end{cases}$$

The schedule begins with original images and then progresses through 2-by-2, 3-by-3, and 4-by-4 permutations; for 7-by-7 and 10-by-10 target runs, it adds a final target-grid stage. Stage permutations use the same deterministic difficulty label as the target record. Each stage processes B scheduled-difficulty views per batch and 20,000 per stage epoch. The stored dataset size does not increase, and validation remains the held-out loader for the final target run.

4.5 Tile Permutation Hardness Metrics

We quantify the difficulty induced by a tile permutation using three complementary, model-agnostic axes and a fourth combined score. Let the image be divided into an $N \times N$ grid. For each source tile i , let (r_i, c_i) denote its original row and column, and let (r'_i, c'_i) denote its destination after applying the permutation.

4.5.1 Adjacency Destruction Hardness

Main idea. This metric asks how much of the original neighborhood structure survives after the tile permutation. If tiles that were next to each other are no longer neighbors,

the image becomes harder for a convolutional model that relies on local structure.

For every original rightward and downward neighboring relation, we test whether the same two source tiles remain adjacent in the same direction after permutation. The number of such canonical adjacency relations is

$$M = 2N(N - 1).$$

If $A_{\text{preserved}}$ is the number of preserved directed adjacencies, adjacency destruction is

$$A = 1 - \frac{A_{\text{preserved}}}{M}.$$

A low value means that much of the original local structure is preserved. A high value means that neighboring tile relations were mostly destroyed.

4.5.2 Global Tile Displacement

Main idea. This metric measures how far tiles move from their original positions. A permutation that only nudges tiles nearby should preserve more global shape than one that sends tiles across the image.

For each tile, we compute its Manhattan displacement in grid coordinates:

$$\delta_i = |r'_i - r_i| + |c'_i - c_i|.$$

The normalized global displacement is

$$D = \frac{\sum_i \delta_i}{D_{\text{max}}},$$

where D_{max} is the maximum possible total Manhattan displacement for an $N \times N$ grid. A low value indicates that tiles remain close to their original locations. A high value indicates large-scale spatial relocation.

4.5.3 Spatial Permutation Entropy

Main idea. This metric asks whether tile movement follows a simple pattern or many different movement patterns. More varied movement directions and distances mean the permutation is less orderly.

For each tile, we compute

$$\Delta_i = (r'_i - r_i, c'_i - c_i)$$

and assign it to a movement bin

$$b_i = (\text{direction}(\Delta_i), \text{distanceTier}(\Delta_i)).$$

The direction is one of

$$\{\text{stationary}, N, S, E, W, NE, NW, SE, SW\}.$$

For non-stationary tiles, the normalized Manhattan distance is

$$d_i = \frac{|r'_i - r_i| + |c'_i - c_i|}{2(N - 1)}.$$

The distance tier is near when $0 < d_i \leq 1/3$, medium when $1/3 < d_i \leq 2/3$, and far when $2/3 < d_i \leq 1$. Stationary tiles use a single stationary bin. If p_b is the empirical probability of movement bin b , the normalized entropy is

$$E = \frac{-\sum_b p_b \log p_b}{\log(N^2)}.$$

A low value means that movement is structured or concentrated in a small number of movement patterns. A high value means that the permutation contains diverse movement directions and distances, making it more disorderly.

4.5.4 Combined Hardness Score

Main idea. The combined score summarizes the previous metrics into one number so permutations can be ranked by an overall notion of difficulty.

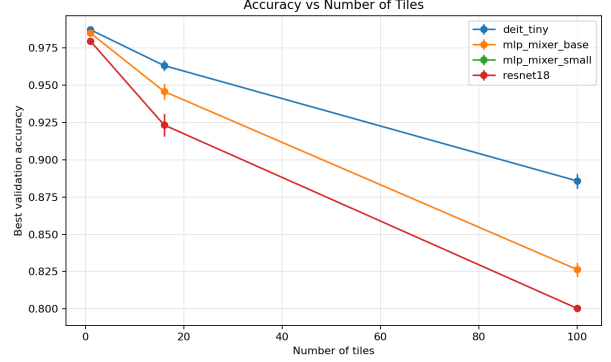


Figure 2: Part 1 validation accuracy as a function of tile count.

The final score is a convex combination of the three normalized components:

$$C = w_a A + w_d D + w_e E,$$

with default weights

$$w_a = 0.5, \quad w_e = 0.3, \quad w_d = 0.2.$$

A low combined score represents an easier permutation with preserved structure, small movement, and low movement disorder. A high combined score represents a harder permutation across these complementary axes.

5 Results

The Part 1 numeric table reflects prior saved outputs and is kept as a reference point until the updated deterministic Colab sweep is rerun. The updated Part 1 design will report the unpermuted baseline labels and the easy, medium, and large deterministic functions for 4-by-4, 7-by-7, and 10-by-10 grids. This supports the proposal’s hypothesis that the Dogs versus Cats image setting is a useful relaxation: the permutation disrupts spatial layout, yet enough visual class information may remain for standard models to learn above chance.

In the prior saved results, Part 2 identifies permutation-difficulty curriculum training as

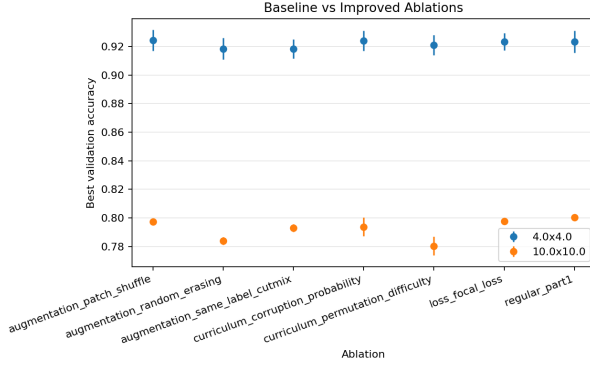


Figure 3: Part 2 comparison of augmentation and curriculum strategies.

the best final 3-by-3 setting, reaching 89.4% mean final validation accuracy and 93.3% mean best validation accuracy. Patch shuffle is the strongest non-curriculum ablation at 87.5%. The updated deterministic matrix will test whether these patterns hold across interpretable easy, medium, and large permutations at larger grid sizes.

Part 3 provides an initial ranking lens for permutations. In the prior joined data, adjacency destruction, spatial entropy, and combined hardness all have negative Pearson correlations with validation accuracy, matching the intuition that destroying local structure and increasing disorder make classification harder. The updated deterministic design increases the joined records to the 4×3 matrix and makes each difficulty label directly comparable across grid sizes.

6 Discussion

The proposal asks whether a neural network can classify post-permutation images and whether performance depends on tile resolution and permutation choice. Prior saved results answer the first question affirmatively for 3-by-3 tile permutations. The updated experiment matrix is designed to answer the

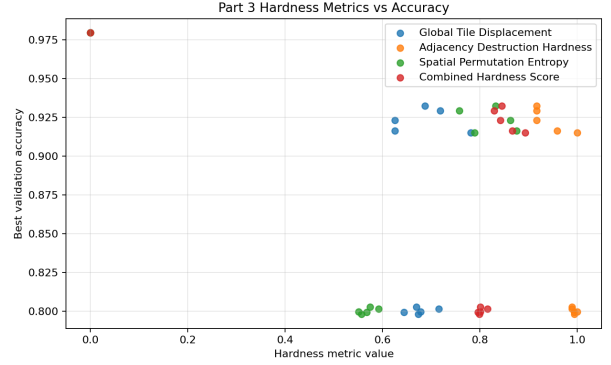


Figure 4: Part 3 metric-versus-accuracy plots for the configured hardness metrics.

same question more systematically across 1 tile / none, 4-by-4, 7-by-7, and 10-by-10 grids, with easy, medium, and large deterministic functions at each grid size.

The prior ablation results indicate that training strategy matters. Patch shuffle and permutation-difficulty curriculum improved over the regular saved baseline, which is consistent with the idea that exposure to spatial disruption can make the model more robust to permuted inputs. The gap between best and final accuracy for the corruption-probability curriculum is an important warning: a method can discover a strong checkpoint while still finishing poorly, so best-checkpoint reporting and final-epoch reporting should both remain in the evaluation.

The hardness metrics address the proposal’s third objective: ranking permutations with the same tile count. The prior evidence is directionally sensible but underpowered. The deterministic matrix improves this by using consistent, named constructions, though repeated seeds and additional architectures will still be needed for stronger statistical conclusions.

7 Conclusion and Future Work

This project implements a reproducible Dogs versus Cats testbed for studying fixed tile-wise image permutations. The work translates the proposal’s motivation into three concrete outputs: baseline classification under permutation, improvement ablations for the permuted setting, and permutation hardness metrics that can be joined to validation accuracy. The updated implementation standardizes all three parts around four grid settings and three deterministic permutation functions, making the next remote Colab sweep directly comparable across baseline, ablation, and hardness analyses.

Future work should expand beyond the deterministic matrix after the updated remote sweep is complete. The next steps are to repeat experiments across seeds, compare more architectures, and add transfer tests between permutation functions. These extensions would better separate memorization of a fixed permutation from direct use of class cues in permuted images, which is one of the central questions posed by the original project proposal.

Part 2 strategy		Training set	Added training exposure	Validation set
Regular baseline	Part 1	Reused Part 1 ResNet-18 runs.	None beyond the fixed target permutation.	Reused Part 1 held-out validation accuracy.
Patch shuffle		Same 20,000 training images and target permutation.	No appended files; each batch of size B produces B shuffled training views, for 20,000 shuffled views per epoch.	Same 5,000 held-out images with the target permutation only.
Random erasing		Same 20,000 training images and target permutation.	No appended files; random erasing is applied with probability 1.0, giving B erased views per batch and 20,000 erased views per epoch.	Same 5,000 held-out images with no random erasing.
Same-label Mix	CutMix	Same 20,000 training images and target permutation.	No appended files; each batch attempts B same-class hybrid views, up to 20,000 hybrid views per epoch.	Same 5,000 held-out images with no CutMix.
Combined augmentations	aug-	Same 20,000 training images and target permutation.	No appended files; each batch receives random erasing plus batch-level CutMix and patch shuffle, yielding B composite views per batch and 20,000 per epoch.	Same 5,000 held-out images with no training-only augmentations.
Focal loss		Same 20,000 training images and target permutation.	No new images; the loss reweights hard examples during optimization.	Same 5,000 held-out images with the target permutation.
Corruption-probability curriculum		Same 20,000 training images at every stage.	No appended files; stage probabilities 0.1, 0.3, 0.5, and 0.8 give expected permuted counts of 2,000, 6,000, 10,000, and 16,000 views per stage epoch.	One fixed target validation loader; no separate validation set per stage.
Permutation-difficulty curriculum		Same 20,000 training images at every stage.	No appended files; each stage processes 20,000 views per epoch at its scheduled difficulty: original, 2-by-2, 3-by-3, 4-by-4, and for 7-by-7 or 10-by-10 targets an additional final target-grid stage.	One fixed final-target validation loader; no separate validation set per stage.

Table 2: Part 2 training and validation sets by ablation. The updated output matrix uses ResNet-18 with 4 grid settings crossed with easy, medium, and large deterministic permutation records, for 12 records per ablation.

Part 2 strategy	Stages	Experiment sequence	Dataset increase / per-epoch views
Regular baseline	Part 1 1	Standard training on the target permutation.	No increase: 20,000 training examples per epoch.
Patch shuffle	1	Standard training, with random patch shuffling applied inside training batches.	0 stored increase; B shuffled views per batch and 20,000 per epoch.
Random erasing	1	Standard training, with random image-region erasing in the training transform.	0 stored increase; B erased views per batch and 20,000 per epoch.
Same-label Cut-Mix	1	Standard training, with same-class patches mixed inside training batches.	0 stored increase; up to B hybrid views per batch and 20,000 per epoch.
Combined augmentations	1	Standard training, with more than one augmentation source active together.	0 stored increase; B composite augmented views per batch and 20,000 per epoch.
Focal loss	1	Standard training with focal loss instead of cross-entropy.	No increase: the examples are unchanged, only their loss weights change.
Corruption-probability curriculum	4	Permutation probability stages: 0.1, 0.3, 0.5, 0.8.	0 stored increase; expected permuted views per epoch are 2,000, 6,000, 10,000, and 16,000 by stage.
Permutation-difficulty curriculum	1, 4, or 5	Baseline target: original only. Tiled targets: original, 2-by-2, 3-by-3, 4-by-4, and the final target grid when larger than 4-by-4, using the same easy/medium/large function as the target record.	0 stored increase; 20,000 scheduled-difficulty views per stage epoch.

Table 3: Part 2 ablation experiment design. Augmentations do not duplicate files on disk; they increase the diversity of training views produced from the same fixed split. Each target record is trained for 10 epochs in the remote Colab configuration.

Part 3 analysis	Training set	Validation source
Hardness metric computation	Not applicable; Part 3 does not train a model.	Reuses Part 1 ResNet-18 validation accuracies.
Metric-accuracy join	Not applicable; only saved CSV outputs are joined.	Matches each metric row to the corresponding Part 1 permutation ID.
Correlation analysis	Not applicable; no parameters are updated.	Computes correlations against the joined held-out validation accuracies.

Table 4: Part 3 data usage. Part 3 analyzes saved permutations and validation accuracies rather than creating new train or validation splits.

Model	Main operation	Inductive bias
ResNet-18	Convolution	Local texture and translation structure
DeiT-Tiny	Self-attention	Patch-to-patch interactions
MLP-Mixer Base	MLP mixing	Patch and feature mixing without attention

Table 5: Architectural comparison of the three frozen feature extractors.

Model	Learnable	Frozen	Total
DeiT-Tiny	386	5,524,416	5,524,802
ResNet-18	1,026	11,176,512	11,177,538
MLP-Mixer Base	1,538	59,111,472	59,113,010

Table 6: Parameter counts for the frozen-backbone comparison with two output classes. “Learnable” counts only the classifier head optimized during training; “Frozen” counts pretrained parameters used only in the forward pass.

Metric	Dimension	What it captures
Adjacency destruction hardness	Local topology	Are original neighbors still neighbors?
Global tile displacement	Global geometry	How far did tiles move from their original position?
Spatial permutation entropy	Permutation disorder	How diverse or irregular are tile movement patterns?
Combined hardness score	Aggregated hardness	Weighted summary of the three normalized axes

Table 7: Tile permutation hardness metrics used in the analysis.

Setting	Tiles	Final Acc.	Runs
Unpermuted	1	86.5%	1
3-by-3 permuted	9	83.7%	2

Table 8: Part 1 ResNet-18 validation accuracy aggregated from saved experiment outputs.

Ablation	Tiles	Final Acc.	Best Acc.	Runs
Regular Part 1 baseline	9	83.7%	83.7%	2
Combined augmentations	9	81.7%	81.7%	2
Patch shuffle	9	87.5%	87.5%	2
Random erasing	9	83.7%	83.7%	2
Same-label CutMix	9	84.6%	84.6%	2
Corruption-probability curriculum	9	78.8%	97.1%	2
Permutation-difficulty curriculum	9	89.4%	93.3%	2

Table 9: Prior Part 2 3-by-3 ablation results for ResNet-18. The updated remote matrix will replace this with easy, medium, and large deterministic records for 1 tile / none, 4-by-4, 7-by-7, and 10-by-10 grids.

Hardness metric	Pearson	Spearman
Global displacement	-0.04	0.50
Adjacency destruction	-0.47	-0.50
Spatial entropy	-0.39	-0.50
Combined hardness	-0.38	-0.50

Table 10: Prior Part 3 correlations between ResNet-18 validation accuracy and permutation metrics, computed over three joined records. The updated analysis joins against the deterministic permutation records.

References

- [1] E. Boyle, J. Holmgren, and M. Weiss. Permuted puzzles and cryptographic hardness. In *Theory of Cryptography*, pages 465–493, 2019.
- [2] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [3] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [4] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy. MLP-Mixer: An all-MLP architecture for vision. In *Advances in Neural Information Processing Systems*, 2021.
- [5] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers and distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021.